```
array(1) {
  ["file"]=>
  array(6) {
    ["name"]=>
    string(10) "random.jpg"
...
array(1) {
  ["animals"]=>
  array(1) {
    ["cat"]=>
    string(32) "jess&humans[]=anna&humans[]=paul"
  }
}
```

As you can see, we have two distinct sets of POST data, as dictated by the multipart `Content-Type:` and `Content-Disposition:` sub-headers. You might think this would result in a `$_POST` array that matches previous examples above where we sent the POST data using curl's `-d` option, but as you can see, the `$_POST` data hasn't actually decoded in the same way. This isn't a PHP problem per se; it's down to how we've used curl. The raw request data created from the above curl command would look something like this:

```
POST /index.php HTTP/1.1
Content-Type: multipart/form-data; boundary=---------DtcO7DDvZ3jzJIDOEPsbxK
--------------------DtcO7DDvZ3jzJIDOEPsbxK
Content-Disposition: form-data; name="file"; filename="random.jpg"
Content-Type: image/jpeg
<BINARY_DATA>
--------------------DtcO7DDvZ3jzJIDOEPsbxK
Content-Disposition: form-data; name="cat";
jess&humans[]=anna&humans[]=paul
```

When creating multipart form requests with curl, it is necessary to provide each `POST` variable in a separate `-F` string, like this…

```
$ curl -F 'file=@./random.jpg' \
  -F 'animals[cat]=jess' \
  -F 'humans[]=anna' \
```